Advancing LLM-Generated Code Reliability: A Hybrid Approach for Hallucination Detection

Bo Yang, Jiayi Dang, Huai Liu, *Senior Member, IEEE*, and Zhi Jin, *Fellow, IEEE*

***Abstract*—The increasing use of Large Language Models (LLMs) for writing code has raised important concerns about "code hallucinations." These occur when the generated code looks correct in terms of its structure (syntax) but contains mistakes in its meaning or logic. Such errors can then spread through software, leading to problems and inefficiencies in the final applications. Current research on finding these code hallucinations in LLM output often struggles with inefficiency. It also lacks a good collection of test cases specifically designed to properly evaluate how well different detection methods work. To address these issues, we introduce a new approach that effectively combines static and dynamic analysis techniques for hallucination detection (SDHD). While standard methods often fail to spot code hallucinations, SDHD shows significant improvement in performance across various datasets. For example, when tested on the MBPP, CodeHaluEval, and HalluCode datasets, SDHD achieved an average precision of 0.771, an average recall of 0.783, and an average F1-score of 0.776. These results are not just slightly better, but substantially higher than those of existing methods, clearly demonstrating SDHD's superior effectiveness in overcoming the limitations of current hallucination detection approaches.**

***Index Terms*—Code generation, large language models, test case generation, hallucination detection.**

I. Introduction

In recent years, LLMs have achieved remarkable success across a multitude of domains, as referenced in the literature [1], [2], [3], [4], ranging from natural language processing [5], [6], [7] to image recognition [8], [9], [10]. These models, characterized by their ability to understand and generate human-like text, have revolutionized the way we interact with machines. A particularly promising area where these models are beginning to make a significant impact is the realm of code generation. Researchers have started to explore the potential of LLMs in automating the process of writing code [11], [12], [13], [14], [15], a task that has traditionally been seen as the exclusive domain of human developers.

The application of LLMs to code generation brings several advantages. Firstly, it has the potential to significantly improve developer productivity by reducing the time and effort required to write code. These models can generate code snippets or even complete programs based on natural language descriptions, allowing developers to focus on higher-level tasks such as design and architecture. Secondly, LLMs can help reduce the likelihood of errors in the code. By generating code that adheres to best practices and standards, these models can help prevent common bugs and vulnerabilities. Lastly, LLMs can support the learning and understanding of programming concepts, especially for beginners who may benefit from seeing examples of well-written code.

There have been several studies and applications that leverage LLMs for code generation. For instance, some researchers have developed models that can automatically generate code for specific tasks, such as creating a web page or analyzing a dataset [11], [13]. Others have created tools that assist developers in writing code by providing suggestions or auto-completing code snippets. These tools can be integrated into development environments, making the writing process more efficient and accessible [13]. Additionally, some studies have focused on generating code that is not only correct but also optimized for performance, taking into account factors such as execution speed and memory usage [12], [13].

Despite the progress made in this area, there are still challenges that need to be addressed. Ensuring the quality and reliability of the generated code is a major concern, as is the need for models that can be understood and adapt to the specific context of a project. Furthermore, there are ethical and legal considerations surrounding the use of LLMs for code generation, such as issues of intellectual property and liability. As the field continues to evolve, it will be important to address these challenges and explore new ways to harness the power of LLMs for code generation. The advent of LLMs has heralded a transformative era in code generation, offering unprecedented capabilities to automate software creation. However, in conjunction with these advances, the issue of hallucination has emerged as a significant concern. Hallucination refers to the generation of code that is syntactically correct but semantically or logically flawed, leading to potential errors and inefficiencies in software applications. Consequently, the detection and

Received 21 July 2025; revised 21 November 2025; accepted 1 December 2025. Date of publication 5 December 2025; date of current version 13 February 2026. Recommended for acceptance by S. Uchitel. *(Corresponding author: Zhi Jin.)*

Bo Yang and Jiayi Dang are with the School of Information Science and Technology, Beijing Forestry University, Beijing 100083, China (e-mail: yangbo@bjfu.edu.cn; dang_jiayi@163.com).

Huai Liu is with the Department of Computing Technologies, Swinburne University of Technology, Hawthorn VIC 3122, Australia (e-mail: hliu@swin.edu.au).

Zhi Jin is with the School of Computer Science, Wuhan University, Wuhan 430072, China (e-mail: zhijin@whu.edu.cn).

Digital Object Identifier 10.1109/TSE.2025.3640641

0098-5589 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

mitigation of hallucinations have become key research areas within the domain of LLM-driven code generation.

The emergence of LLMs has ushered in a revolutionary phase in the domain of code generation, presenting unparalleled opportunities to automate the process of software creation. However, alongside these advancements, the issue of code hallucination has risen as a critical concern. Code hallucination refers to instances where generated code, while syntactically correct, contains semantic or logical errors, which can lead to potential malfunctions and inefficiencies in software applications. As a result, the detection and mitigation of these hallucinations have become pivotal areas of research within the field of LLM-driven code generation.

The current research landscape is marked by several significant contributions, particularly studies that explore the categorization of hallucinations [16], their prevalence across various LLMs [17], and the mechanisms behind their occurrence [18]. These studies offer novel insights into the identification and analysis of hallucinations in code produced by LLMs. Tian et al. [17] introduced the concept of code hallucinations and proposed a classification scheme based on execution verification. They categorized hallucinations into four primary types: mapping, naming, resource, and logic hallucinations, each with its subcategories. The study also introduced CodeHalu, a dynamic detection algorithm designed to identify and quantify code hallucinations, along with the CodeHaluEval benchmark, which includes a diverse set of tasks and samples to evaluate hallucinations in LLMs. Liu et al. [16] provided a thorough analysis of the types of hallucinations that LLMs may produce in code generation and established a taxonomy of hallucination types. It examined the distribution of hallucinations and their correlation with code correctness. Furthermore, the authors proposed HalluCode, a benchmark for assessing the performance of code LLMs in recognizing hallucinations. Zhang et al. [18] focused on hallucinations within more practical and complex development contexts, such as repository-level generation scenarios. It established a taxonomy for hallucinations in LLM-based code generation and explored the phenomenon, mechanisms, and mitigation strategies for LLM hallucinations. The authors also proposed a retrieval-augmented generation (RAG) approach to mitigate hallucinations in LLM-based code generation.

Collectively, these studies represent substantial progress towards understanding and addressing hallucinations in LLM-generated code. However, existing research on detecting code hallucinations in LLM-generated code is lacking in efficiency. The potential of static analysis, which can identify issues without executing the code, remains underutilized. Moreover, the absence of comprehensive test suites for evaluating different approaches hinders the ability to compare and benchmark their effectiveness. To enhance the detection of code hallucinations, a more holistic approach that integrates both dynamic and static analysis, and employs domain-specific static analyzers, is essential. Additionally, the development of test suites is critical for evaluating and advancing the field.

In response to these challenges, we introduce SDHD, a hybrid framework that combines static analysis and dynamic execution for detecting hallucinations in LLM-generated code. This comprehensive framework is specifically designed to identify and address code hallucinations in LLM-generated code. SDHD integrates both static analysis and dynamic testing, employing SSA forms for static analysis and a differential prompting technique for dynamic testing. This comprehensive approach allows for the detection of a broader range of code hallucinations, including both syntactic and semantic errors. SDHD enhances test case generation through a differential prompting technique, prompting the LLM to generate multiple versions of the code for the same input, thereby increasing test coverage and identifying issues that may not be captured by a limited set of test cases. Furthermore, SDHD is better equipped to detect logical errors in the code, with the dynamic testing component allowing for the execution of code and the observation of its behavior, crucial for identifying logical issues that cannot be detected through static analysis alone.

The contribution of the paper is three-fold, as summarised in the following:

- Combined Static Analysis and Dynamic Execution: SDHD incorporates both static analysis and dynamic testing, using Static Single Assignment (SSA) forms for static analysis and a differential prompting technique for dynamic testing. This comprehensive approach allows for the detection of a broader range of code hallucinations, including both syntactic and semantic errors.
- Enhanced Test Case Generation: SDHD employs an innovative technique called differential prompting to enhance test case generation for LLM-generated code. Instead of relying on a single test case for each input, it prompts the LLM to generate test cases of the code. This approach effectively increases test coverage by exploring a wider range of potential behaviors and outcomes.
- Experimental Results: SDHD has shown superior performance in detecting code hallucinations compared to exiting state-of-the-art approaches. The experiment highlights that SDHD achieves excellent detection results on three datasets.

The remainder of this paper is organized as follows. Section II briefly introduces the background of SDHD. Section III presents our proposed SDHD. Section IV reports the empirical study for the evaluation of SDHD and Section V describes the experiment's result. Section VII briefly introduces the related work of hallucination detection. Section VIII summarizes the paper.

II. Background

A. A Motivation Example

To better illustrate the limitations of existing natural language processing hallucination detection and dynamics-based execution of hallucination detection, we give further detailed examples in this section.

As depicted in Fig. 1, part (a) illustrates the application of a traditional natural language processing domain using a text similarity approach to detect hallucinations. It can be observed that the generated code has a ROUGE score of 0.3684 when compared to the benchmark code. This result suggests that the

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

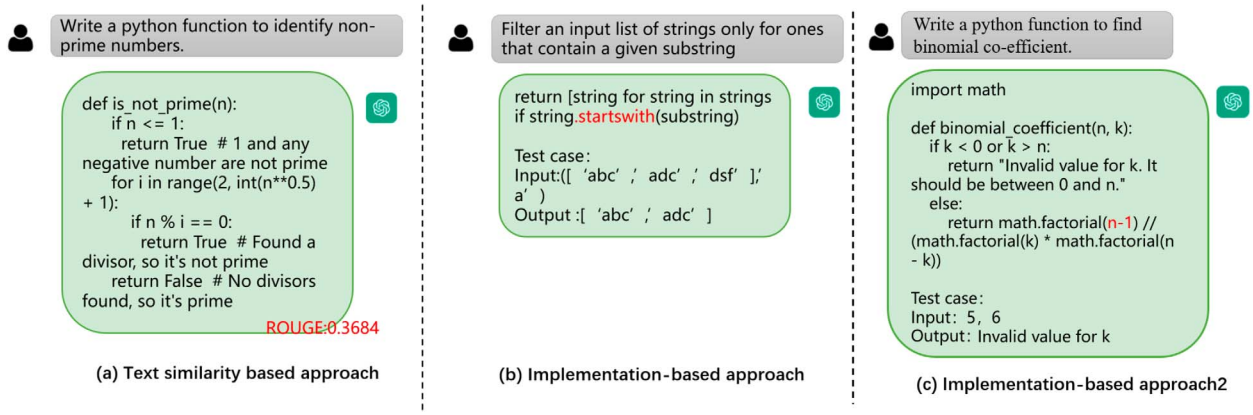


Fig. 1. A motivation example.

generated code has a low similarity to the benchmark. However, a careful analysis of this code reveals that it meets the user's input requirements, indicating that the code generated by the LLM is compliant.

Part b of Fig. 1 employs the only current detection method for code-generated hallucination detection—execution-based code detection. It can be seen that the existing test cases are passed successfully, appearing to meet the user's intent. Yet, a closer examination of the code reveals that the user's original intent was to filter out strings containing a given character. The generated code, however, filters out strings that start with the given character, a discrepancy that the provided test cases fail to detect. Thus, the coverage of the test cases significantly impacts the quality of code hallucination detection.

Furthermore, issues such as variables declared but never used, undefined variables utilized but not covered by the executed test cases, mismatches in variable or function parameter types, code segments that are never executed due to incorrect conditional statements, and statement errors not executed by the test cases, cannot be adequately detected by execution-based dynamic testing. This is demonstrated in Fig. 1, part c. The given test cases do not execute the erroneous code segment, highlighting the necessity to incorporate a static code detection approach, as shown in Fig. 1, part (c).

B. Code Generation Hallucination

In the realm of code generation tasks, the phenomenon of hallucinations has garnered significant attention. Tian et al. [17] were pioneers in systematically defining and categorizing this phenomenon in their research. classified code hallucinations into four primary types: Mapping Hallucination, Naming Hallucination, Resource Hallucination, and Logical Hallucination. Below is a detailed elaboration of these four categories.

Mapping Hallucination (MH) occurs when LLMs exhibit blurred or confused perceptions and mappings of data types, values, and structures during data manipulation. This type of hallucination is further subdivided into data compliance hallucination and structure access hallucination.

Data compliance hallucination (DCH) arises when the model fails to correctly handle data type or format requirements during code generation, resulting in code that is incompatible with input data. For instance, the model might erroneously map string data to integer types, leading to runtime errors.

Structure access hallucination (SAH) occurs when the model makes errors while accessing complex data structures (such as arrays, dictionaries, or trees), resulting in code that cannot correctly read or modify data. For example, the model might incorrectly access a non-existent array index, causing exceptions.

Naming Hallucination (NH) pertains to memory issues or factual inaccuracies related to the naming, scope, and existence of variables, attributes, and modules in the code generated by LLMs. This type is further divided into identity hallucination and external source hallucination.

Identity hallucination (IH) occurs when the model fails to correctly identify or reference the identity information of variables, functions, or modules during code generation, leading to code that cannot execute correctly. For example, the model might incorrectly reference an undefined variable, causing runtime errors.

External source hallucination (ESH) arises when the model makes errors in referencing external libraries or modules, resulting in code that cannot correctly call external resources. For instance, the model might incorrectly reference a non-existent library function, leading to compilation errors.

Resource Hallucination (RH) refers to the model's lack of adequate perception and prediction of resource consumption and control flow during actual execution, leading to issues in resource management in the generated code. This type is further categorized into physical constraint hallucination and computational boundary hallucination.

Physical constraint hallucination (PCH) occurs when the model fails to consider the physical limitations of hardware resources (such as memory size, CPU performance, etc.) during code generation, resulting in code that exceeds resource limits at runtime. For example, the model might generate an algorithm that consumes excessive memory, leading to memory leaks or program crashes.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

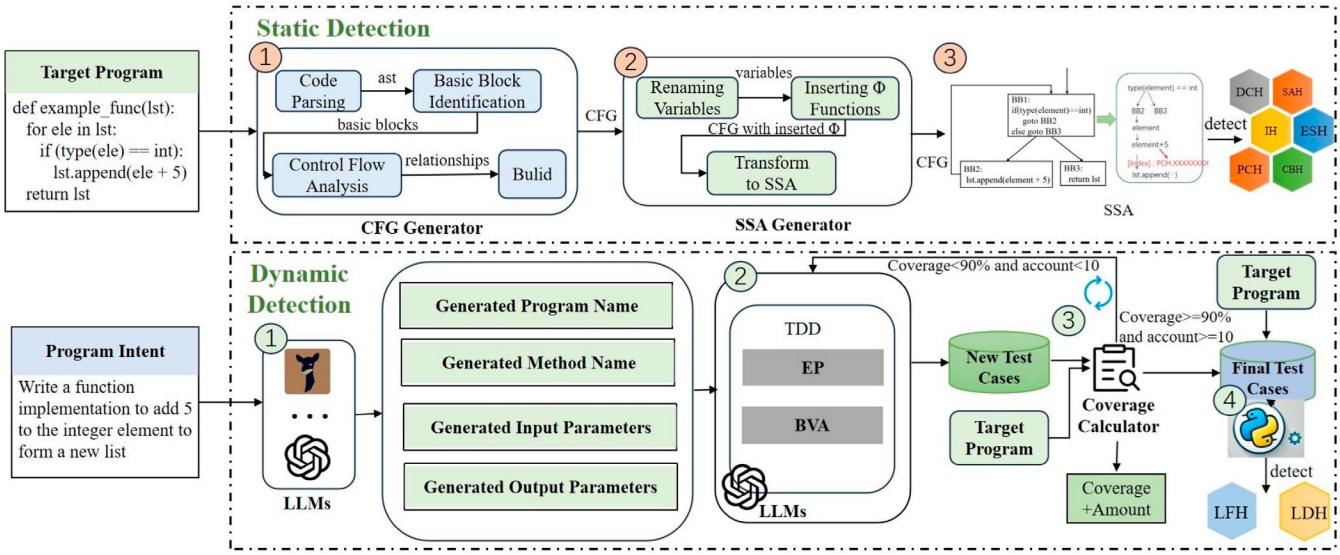


Fig. 2. The hallucination detection framework.

Computational boundary hallucination (CBH) arises when the model fails to correctly handle computational boundary conditions (such as loop iterations, recursion depth, etc.) during code generation, resulting in code that enters infinite loops or stack overflows at runtime. For instance, the model might generate a recursively functions that does not terminate correctly, causing program crashes.

Logical Hallucination (LH) encompasses situations in which the expected outcomes diverge from the actual results following the execution of LLM-generated code, or where the output exhibits low semantic density or complete incoherence. This phenomenon is further categorized into logical deviation hallucination and logical failure hallucination.

Logical deviation hallucination (LDH) manifests when the model inadequately comprehends task requirements during code generation, producing code with minor logical flaws that nevertheless retains partial functionality. One typical example for LDH is that a sorting algorithm generates output lacking complete ordering.

Logical failure hallucination (LFH) emerges from the model's complete misinterpretation of task requirements during code generation, yielding code that is fundamentally flawed, non-executable, or generates meaningless results. For example, the model might produce an incorrect mathematical formula, resulting in calculations that are entirely inconsistent with expected outcomes.

III. Methodology

The SDHD, as illustrated in Fig. 2, is primarily composed of two distinct components: static detection and dynamic detection. The static detection module takes the code generated by LLMs as input and outputs the results of hallucination detection. Initially, the source code is tokenized into a series of elements, including identifiers, keywords, operators, and separators. Subsequently, an Abstract Syntax Tree (AST) is constructed using these lexical units. The AST is then traversed to perform semantic analysis, which includes type checking, variable declarations, and other related checks. This process also involves collecting information on variable definitions and uses, leading to the creation of a symbol table. Following the AST analysis, control flow information is extracted to construct a Control Flow Graph (CFG), where nodes represent basic blocks and edges denote control flows. Data flow analysis is conducted on the CFG to identify active variables at each program point. Each variable is assigned a unique version number, and the points of definition and use for each variable are determined. This facilitates the construction of SSA form, which is then utilized for detecting code-generated hallucinations. Static detection is effective in identifying the first six types of hallucinations, namely DCH, SAH, IH, ESH, PCH, and CBH.

The dynamic detection module, on the other hand, takes natural language descriptions of program intent as input and outputs the results of hallucination detection. Initially, the requirements are refined to extract method names, input parameters, and output parameters for code generation. Equivalent class partitioning and boundary value analysis methods, commonly used in black box testing, are then employed with the assistance of LLMs to generate new test cases. The number of test cases is calculated until they meet the desired criteria. Finally, the generated test cases are used to drive the execution of the generated code, thereby detecting hallucinations in the code generation process based on the execution results. The detection of these hallucinations enables developers to refine and improve the code, ensuring that it meets the desired specifications and functions reliably in real-world scenarios. Dynamic execution is utilized for the detection of LDH and LFH.

A. Constructing the SSA of a Program

Constructing an AST: The construction of an AST for a given Python program as shown in Fig. 3 during the SSA construction

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

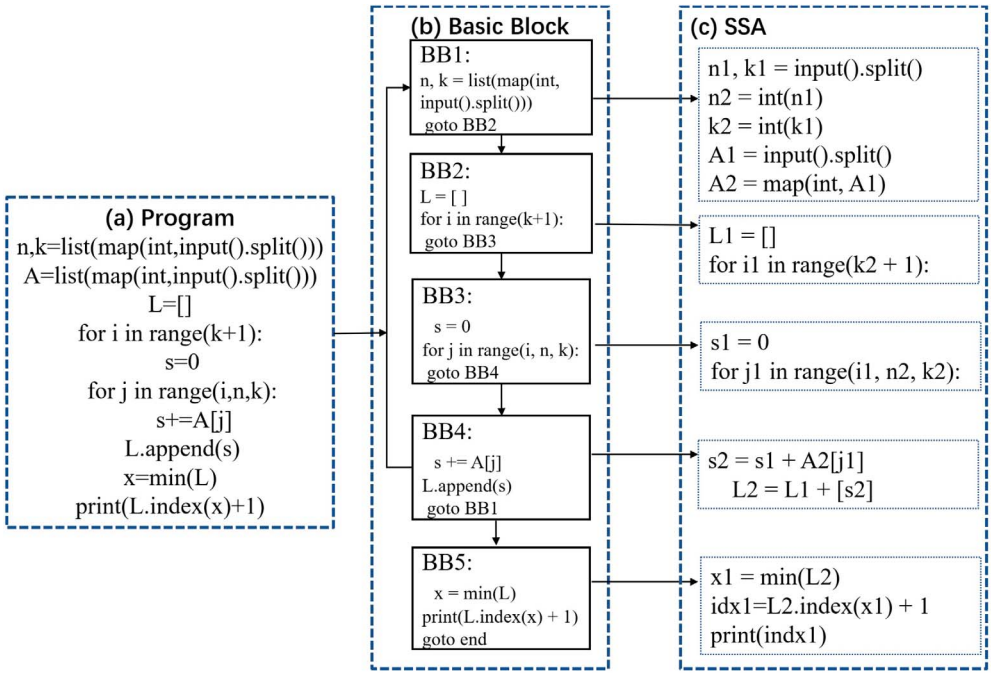


Fig. 3. Constructing the SSA of a program.

process can be outlined in the following steps. The source code is initially tokenized into a sequence of tokens, which are subsequently parsed to form an AST. The AST encapsulates the program's structure, with nodes corresponding to expressions, statements, and control flow constructs. In this specific case, the AST would encompass nodes for variable declarations (e.g., *n, k, A, L*), loops, conditional statements, and arithmetic operations.

To facilitate a precise and formal description of our methodology, we first introduce the notation that will be used throughout this section. We denote the set of all valid Python source code strings as $\mathcal{S}$. The process of lexical analysis transforms a source code string $s \in \mathcal{S}$ into a sequence of tokens, which we represent as an element of the set $\mathcal{T}_{seq}$. Subsequently, syntactic analysis parses this token sequence into an Abstract Syntax Tree (AST). The AST is formally defined as a directed graph $G = (N, E)$, where N is the set of nodes and E is the set of directed edges. Each node $n \in N$ corresponds to a syntactic construct. Furthermore, to explicitly track variable assignments, we introduce a special set of definition nodes, denoted by $D \subseteq N$. This foundational notation allows us to rigorously define the transformations involved in AST construction.

To express this method with greater clarity and formality, we first define the AST construction as a composite transformation function

$$\text{ConstructAST} : \mathcal{S} \to \mathcal{T}_{AST} \tag{1}$$

where $\mathcal{T}_{AST}$ is the set of all possible Abstract Syntax Trees. This function is decomposed into two primary phases: (1) Lexical Analysis, a mapping $L : \mathcal{S} \to \mathcal{T}_{seq}$ that transforms source code into a token stream, and (2) Syntactic Analysis, a parsing function $P : \mathcal{T}_{seq} \to \mathcal{T}_{AST}$ that constructs the tree. The resulting tree is formally represented as a directed graph $G = (N, E)$, where the node set N is defined as

$$N \subseteq \{\text{Module}, \text{For}, \text{Assign}, \text{Var}, \text{BinOp}, \text{Call}, \dots\} \tag{2}$$

Algorithm 1 AST Construction with Definition Nodes

Require: Python source code S
Ensure: An AST $G = (N, E)$ with definition nodes D
1: $T_{seq} \leftarrow \text{LexicalAnalysis}(S)$
2: $G \leftarrow \text{Parse}(T_{seq})$
3: $D \leftarrow \emptyset$ ▷ Initialize set of definition nodes
4: **for** each node $n \in N$ **do**
5: **if** n.type $=$ Assign **then**
6: $d \leftarrow \text{CreateDefinitionNode}(n.\text{target})$
7: $D \leftarrow D \cup \{d\}$
8: $E \leftarrow E \cup \{(n, d)\}$ ▷ Link definition to assignment
9: **end if**
10: **end for**
11: **return** G annotated with D

Algorithm 1 presents the pseudocode for this process, which systematically builds the tree and annotates it with variable definition nodes.

The process begins with the lexical analyzer (Algorithm 1, line 1), which breaks down the source code into a sequence of tokens. Each token signifies a syntactic component of the code, such as identifiers (e.g., *n, k, A, L*), literals (e.g., 1), operators (e.g., =, +, -, *, /), and delimiters (e.g., .). Following this, the parser (line 2) takes the token stream and constructs the AST by identifying patterns that adhere to the grammar rules

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

of the Python language. This involves creating nodes for each syntactic construct:

1) A *For* node is generated for the loop `for i in range(k+1):`, which includes the nested loop and the append operation in its body.
2) Within the outer loop, a Variable node is created for *s*, initialized to 0.
3) Another *For* node is established for the inner loop `for j in range(i, n, k):`.
4) Inside the inner loop, an *Assignment* node is formed for `s += A[j]`, representing the addition operation and the update of the variable *s*.
5) An *Append* node is created for `L.append(s)`, symbolizing the addition of the sum to the list *L*.

The AST hierarchically represents the program's structure. The root of the AST is a Module node, which contains a sequence of statements. For this program, the AST features a For loop as the primary statement, with its body consisting of another For loop, an Assignment statement, and an Append operation. This hierarchical structure can be formally represented as follows, providing a clear graphical expression of the final output.

As the parser constructs the AST, it also monitors variable definitions (Algorithm 1, lines 4-8). Each time a variable is defined (assigned a value), a new Definition node is created in the AST. For this program, this would include nodes for *n, k, A*, and *L* at the beginning, and for *s* within the loops. This annotated AST serves as the structured foundation for the subsequent SSA construction phase.

Constructing a CFG: Upon constructing the AST for the given Python code, the initial phase in developing the Control Flow Graph (CFG) involves segmenting the AST into basic blocks. Each basic block is a contiguous sequence of code with no internal branches, except for entry and exit points.

We establish the entry and exit nodes initially. The CFG commences with an entry node, symbolizing the program's inception. This entry node is connected to the first basic block, which, in this context, encompasses input parsing and list initialization. Subsequently, nodes are created for loops and conditional statements. The provided code contains two nested loops, each represented by a distinct node in the CFG. The outer loop node connects to the inner loop node and to the node responsible for appending the sum *s* to the list L. The inner loop node includes a back edge for loop continuation and an exit edge to the node that increments the loop variable i.

Following this, basic blocks are interconnected based on the program's control flow. For instance, an edge is established from the node initializing *s* to 0 to the inner loop node. After the inner loop, an edge leads to the node appending *s* to *L*. Post appending, an edge either proceeds to the next iteration of the outer loop or to the node calculating the minimum value *x* if the loop condition is not met.

Finally, termination nodes are added. The CFG culminates with a termination node, signifying the program's conclusion. In this code, the termination node succeeds the node printing the index of the minimum value *x*. All paths not leading to another loop iteration or conditional branch terminate at this node, thereby completing the CFG representation of the program's control flow.

Constructing an SSA: To systematically construct the SSA form, we first clarify the foundational concepts and notation. The process operates on the program's Control Flow Graph (CFG), a directed graph $G_{CFG} = (N, E)$ where nodes N represent basic blocks of code and edges E represent control flow between them. The construction of SSA form involves two primary tasks: (1) **Variable Renaming,** where each assignment to a variable v is given a unique new name, such as $v_1, v_2, \ldots$, to enforce the single-assignment property; and (2) **Merging Values at Convergence Points,** where different versions of a variable from distinct control flow paths must be reconciled. To fulfil the second task, we introduce the ϕ function. A ϕ function, placed at the beginning of a node where multiple paths merge, acts as a pseudo-function that selects the appropriate version of a variable based on the path taken at runtime. For instance, an instruction like $x_3 = \phi(x_1, x_2)$ implies that x_3 will take the value of x_1 if execution arrives from one predecessor, and x_2 if it arrives from another. The strategic placement of these ϕ-functions is guided by a graph-theoretic concept known as the dominance frontier, which we will formalize shortly.

The construction of the SSA form for the given Python program involves transforming the code into a representation where each variable is assigned exactly once. This process includes identifying all definitions and use points for variables and tracking different versions of each variable. For variables defined on multiple control flow paths, ϕ functions are inserted at the nodes where these paths converge. A ϕ function is a special operator in SSA that selects the correct version of a variable from different paths. For example, at the point where two paths merge, we insert $x_3 = \phi(x_1, x_2)$, where x_1 and x_2 are versions of the variable from different paths. In SSA form, each assignment to a variable is renamed to a new unique identifier. For instance, if x is assigned multiple times in a program, the variable after each assignment might be renamed to x_1, x_2, x_3, etc. This process ensures the single assignment property of SSA.

To formalize this method, we define the SSA construction as a transformation on the program's CFG. The placement of ϕ functions is determined by the dominance frontier of a node. For a node n, its dominance frontier $DF(n)$ is the set of all nodes m where n dominates a predecessor of m but does not strictly dominate m itself. A ϕ function for a variable v must be placed at every node in the union of the dominance frontiers of all its definition sites.

Algorithm 2 presents the pseudocode for our SSA construction method, which follows the classic algorithm by Cytron et al. This algorithm systematically fulfils the two core tasks: inserting ϕ-functions and renaming variables.

The application of this algorithm to our program is detailed in the following steps. Firstly, we rename variables. The initial step in SSA construction (Phase 2 of Algorithm 2) is to rename variables so that each variable is assigned only once. This involves creating a new name for each variable every time it is assigned a new value. For the given program, we start by renaming n and k to n_1 and k_1 respectively after their initial assignment.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 SSA Construction

Require: $CFG = (N, E)$, Set of variables V
Ensure: CFG in SSA form
Phase 1: ϕ-function Insertion
for each variable $v \in V$ **do**
  $DefSites(v) \leftarrow \{n \in N \mid v \text{ is defined in } n\}$
  $WorkList \leftarrow DefSites(v)$
  while $WorkList \neq \emptyset$ **do**
    $n \leftarrow WorkList.pop()$
    for each $y \in DF(n)$ **do**
      if y has no ϕ-function for v **then**
        Insert ϕ-function for v at the start of y
        $WorkList.push(y)$
      end if
    end for
  end while
end for
Phase 2: Variable Renaming
`RenameVariables`(EntryNode of CFG) ▷ Recursive traversal
return CFG in SSA form

TABLE I
VARIABLE RENAMING IN SSA FORM

Original Variable	SSA Versions	Definition Point
n	n_1	Initial assignment
k	k_1	Initial assignment
A	A_1	List comprehension
i	$i_2, i_3, \ldots$	Outer loop iterations
s	$s_2, s_3, \ldots$	Inside the outer loop
j	$j_3, j_4, \ldots$	Inner loop iterations
x	x_4	Assignment from `min(L)`

Similarly, A becomes A_1 after the list comprehension. Table I provides a comprehensive mapping of the original variables to their SSA versions, making the renaming process explicit and easy to follow.

Secondly, we insert ϕ functions. Since the program contains a loop, we need to insert ϕ nodes at the beginning of the loop to merge the incoming values from different paths. However, in this simple program, there are no variables that need to be merged across different iterations, so no ϕ functions are required. Next, we convert the loop iteration to SSA. For each iteration of the loop, we create new variables for i and s. The loop variable i is renamed to i_2, i_3, . . ., for each iteration. Inside the loop, the variable s is assigned a new value in each iteration, so we rename it to s_2, s_3, . . ., for each assignment.

Then, we convert the array access to SSA. The array access $A[j]$ is replaced with $A_1[j_3]$ in the SSA form. Since j is also a loop variable, it is renamed to j_3, j_4, . . ., for each access within the inner loop. Finally, we convert the final assignments to SSA. The final assignments outside the loop, such as $x = \min(L)$ and `print(`$L.index(x_4)$ `+ 1)`, are also converted to SSA form. The variable x is renamed to x_4, and the access to $L.index(x)$ is replaced with $L.index(x_4)$.

It is important to note that static analysis excels at identifying certain classes of logical hallucinations—such as infinite-loop errors induced by control-flow distortions, while dynamic analysis is more effective at capturing semantic hallucinations, including input-dependent inconsistencies that only manifest at runtime. However, static techniques have limited capabilities in detecting latent or fine, grained semantic defects, and dynamic execution is inherently insufficient for uncovering issues tied to partial, coverage or environment, sensitive behaviors. These complementary limitations underscore the necessity of combining static and dynamic methods to achieve comprehensive and robust hallucination detection.

B. Static Hallucination Detection

The application of the SSA form in detecting hallucinations within LLMs involves a detailed analysis of the information flow through the model's computational graph.

Initially, the computational operations of the language model are represented in SSA form. This transformation involves converting the model's operations into a sequence where each variable is assigned exactly once. The unique assignment property of SSA simplifies the tracking of data origin and propagation within the model. Subsequently, SSA is utilized for tracking data flow. In SSA form, the data flow through the model's layers can be closely monitored. Each variable's value is derived either from the input data or from other variables assigned based on the model's weights and biases. By tracking these assignments, we can pinpoint where specific information is generated or altered within the model. Then, hallucinations are identified using SSA. In the context of language models, a hallucination occurs when the model generates information not supported by the input data. Using SSA, we can detect this by identifying variables that are assigned values without a clear, traceable connection to the input or previously established facts. This could manifest as assignments not directly influenced by the input data or variables assigned based on the input data.

Furthermore, anomalies in the program can be detected. By analyzing the SSA graph, we can develop heuristics or statistical methods to identify anomalies in the data flow. For example, if a variable's value significantly deviates from what is expected based on the input and the model's learned parameters, this could indicate a hallucination. Such instances can be flagged for further review.

For instance, we can detect Structural Access Hallucination (SAH) in the program. The checkpoint is to verify that the access to an array or list does not exceed its boundaries. We find that within the inner loop, the index *j* iterates over a range defined as the range(*i1, n2, k2*). A step size of *k2* equal to zero would result in an invalid step error. If *k* equals zero, the range function will throw a ValueError, which falls under the category of SAH.

C. Dynamic Hallucination Detection

Static hallucination detection methods can identify certain types of code-generation hallucinations produced by LLMs. However, these methods are ineffective in detecting logical

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 Dynamic Hallucination Detection

Require: UR, LLM-generated code C, Max iterations I_{max}
Ensure: Hallucination detection result R_{detect}
1: $TestCases \leftarrow \emptyset$
2: $i \leftarrow 0$
3: **Phase 1: Requirement Analysis**
4: $Prompt_{req} \leftarrow$ CreateRequirementPrompt(UR)
5: $Requirements \leftarrow$ LLM$(Prompt_{req})$
6: **Phase 2: Test Case Generation**
7: **while** $i < I_{max}$ **do**
8: $Prompt_{test} \leftarrow$ CreateTestPrompt$(Requirements)$
9: $TestCases_{new} \leftarrow$ LLM$(Prompt_{test})$
10: $TestCases \leftarrow TestCases \cup TestCases_{new}$
11: **Phase 3: Test Case Evaluation**
12: $Coverage \leftarrow$ EvaluateCoverage$(TestCases)$
13: **if** $Coverage \geq C_{min}$ **then**
14: **Phase 4: Dynamic Execution**
15: $Result \leftarrow$ ExecuteCode$(C, TestCases)$
16: **if** $Result$ is PASS **then**
17: $R_{detect} \leftarrow$ NO_HALLUCINATION
18: **return** R_{detect}
19: **end if**
20: **end if**
21: **Phase 5: Refinement**
22: $Feedback \leftarrow$ GenerateFeedback$(Result, Coverage)$
23: $Requirements \leftarrow$ Refine$(Requirements, Feedback)$
24: $i \leftarrow i + 1$
25: **end while**
26: $R_{detect} \leftarrow$ POTENTIAL_HALLUCINATION
27: **return** R_{detect}

hallucinations, where the code is syntactically correct but semantically incorrect. To address this limitation, we propose a dynamic execution approach for detecting logical hallucinations. Dynamic execution, however, requires the availability of test cases, which are typically not available until after the code has been generated. To address this challenge, we have developed a test case generation method grounded on the principles of Test-Driven Development (TDD). The overall workflow is formalized in Algorithm 3, which outlines the iterative process of test case generation, execution, and refinement.

The SDHD method begins with **Phase 1: Requirement Analysis**, as detailed in lines 4-5 of Algorithm 3. In this initial step, we utilize an LLM to capture the user's requirements (UR), including parameters and function names. This information is then used to create a prompt ($Prompt_{req}$) that guides the LLM in generating a structured set of requirements, which will serve as the foundation for subsequent test case generation.

Following the analysis, the method enters an iterative loop, starting with **Phase 2: Test Case Generation** (lines 8-10). Here, we craft a prompt ($Prompt_{test}$) that includes specific instructions for the LLM to generate a Python function that meets certain criteria. For example, the function should have a descriptive name, accept an integer list as input, return a new integer list, and be memory-efficient. Since no existing code is available at this stage, the focus is on black-box testing. The LLM is guided by equivalence class partitioning and boundary value analysis methods to generate test cases. These test cases cover various scenarios, including an empty list, a list with a single element, a list with the maximum integer value, and a list with non-integer elements.

Once the test cases are generated, we proceed to **Phase 3: Test Case Evaluation** (lines 11-12). The generated test cases are evaluated to ensure they meet a specific coverage criteria. We define coverage as the number of unique test cases, formally $Coverage = |TestCases|$. The process continues only if $Coverage \geq C_{min}$ (e.g., $C_{min} = 10$), which ensures a comprehensive testing process and prevents the method from proceeding with an insufficient test suite.

If the coverage criteria are met, we advance to **Phase 4: Dynamic Execution** (lines 13-17). The generated test cases are then used to dynamically execute the code (C) generated by the LLM. This aids in detecting logical hallucinations, where the code may be syntactically correct but semantically incorrect. If the code passes all tests, we conclude there is no hallucination and terminate the process, returning a successful result.

If the code fails any test or the coverage is deemed insufficient, the method enters **Phase 5: Refinement** (lines 18-20). Feedback is generated based on the execution result and coverage metrics. This feedback is used to refine the requirements for the next iteration of test case generation. The process is iterative, with the LLM continuously refining the test cases based on feedback from the dynamic execution. This enhances the accuracy of the test cases and the overall quality of the generated code.

By adhering to these steps, our proposed method leverages TDD principles to generate test cases for detecting hallucinations in code generated by LLMs. This approach ensures that the generated code is not only syntactically correct but also semantically accurate, thereby reducing the likelihood of hallucinations.

D. The Static and Dynamic Hallucination Detection Result

For code segments with hallucinations, we will output an error message. This error message primarily contains eight types of hallucinations, which include six static hallucination detection results and two dynamic hallucination detection results. The six static hallucination detection results are DCH, SAH, IH, ESH, PCH, PCH, and CBH. In addition, the two dynamic hallucination detection results are Logic Deviation Hallucinations (LDH) and Logic Breakdown Hallucinations (LBH). The specific descriptions of these two hallucinations are as follows:

1) Logic Deviation Hallucinations (LDH): They occur when the code generated by the LLM lacks sufficient logical considerations or contradicts the expected instructions. This hallucination typically does not result in errors during code execution; however, the program's output fails to meet the expected outcome due to logical deviation or confusion.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DETAILED STATISTICS ON DATASETS

Benchmark	Year	Language	Construction	# of Tasks	Avg. Tests/Task	Avg. Tokens/Sample
MBPP	2021	Python	Manual	974	3.0	24.2
CodeHaluEval	2024	Python	Automated	699	12.8	293.2
HalluCode	2024	Python	Automated+Manual	5664	N/A	54.1

2) Logic Breakdown Hallucinations (LBH): This occurs when the LLM struggles to explain or maintain a continuous understanding of the context during code generation. This suggests that the model may lose its way in generating code, making it difficult to maintain strict control over the persistence of contextual information.

E. The Hallucination Detection Result

Static and dynamic detection methods both produce results. It is necessary to deduplicate the results obtained from both static and dynamic detection. Let Q represent the user requirements input to the large language model, A represent the code output by the LLM, and SD represent the detection method used by SDHD. Let H be the set of all hallucination information that can be detected, $H = \{h_1, h_2, \ldots, h_i, \ldots, h_n, \ldots\}$, where h_i represents the i-th piece of hallucination information. $h_i = \{l_i : m_i\}$, where l_i represents the location of the hallucination, and m_i represents the specific content of the hallucination.

Assume that this hallucination information comes from the result sets O_1 and O_2 of static and dynamic detection, respectively. Therefore, we obtain the final result D_Output as follows

$$D_Output = \{h_j | h_j \in SD(A(Q))\} = O_1 \cup O_2$$

IV. EXPERIMENTS

A. Datasets

We constructed the evaluation dataset. For all the LLMs interactions involved in the detection process, we used GPT3.5[1], directly using the interface provided by openAi. We gathered existing datasets related to code generation and, based on these datasets. The datasets are presented in Table II.

Table II summarizes key characteristics of the datasets used in our study. Specifically, for each benchmark, we list its publication **Year** and the primary **Language**. The **Construction** column describes the methodology used to create the dataset: "Manual" means human-crafted data, "Automated" refers to data generated by scripts or models, and "Automated+Manual" denotes a hybrid approach. We also report the total **# (number) of Tasks** in each dataset, the average number of test cases per task (**Avg. Tests/Task**), and the average number of tokens per sample (**Avg. Tokens/Sample**) to provide insight into their scale and complexity. HalluCode, being manually curated for hallucination classification rather than functional testing, does not include executable test cases, hence its "Test" entry is marked as N/A.

[1]https://platform.openai.com/docs/api-reference/chat

MBPP [19]: The MBPP dataset constructed by Google researchers, which allied to evaluate and enhance the performance of programming models on basic Python programming tasks. It comprises 974 programming tasks designed by entry-level programmers to synthesize short Python programs through natural language descriptions. Each task includes a description of a specific problem, a Python function to solve the problem, and three test cases to verify the correctness of the function. These test cases are written in the form of assertion (assert) statements to ensure the code's correctness upon execution.

CodeHaluEval (CHE) [17]: This dataset was generated using the dynamic detection algorithm CodeHalu. Hallucinations are categorized into four main categories and eight subcategories for comparing various hallucinations and their frequencies in different LLM code generations. The development of the CodeHalu benchmark, based on the APPS test set, follows a structured process of validation - identification-construction, providing a standardized methodology for identifying code hallucinations and offering in-depth analysis and improvement of LLMs' performance in code generation accuracy.

Test: average number of test cases shipped with each problem; Tokens: average character-length of the reference solution. HalluCode is manually annotated for hallucination categories rather than functional correctness, hence Test is marked N/A.

HalluCode (HC) [16]: This dataset is based on HumanEval and DS-1000, generated on LLMs CodeGen [20], CodeRL [21], and ChatGPT (GPT-3.5-turbo version), and constructed through manual annotation by the researchers. The hallucination classification includes five types: Intent Conflicting, Context Deviation-Inconsistency, Context Deviation-Repetition, Knowledge Conflicting, and Context Deviation-Dead Code.

B. Research Questions

To evaluate the effectiveness of SDHD and compare it with several state-of-the-art methods on specific tasks, our experiments were specifically designed to address the following four research questions:

RQ1: What is the effectiveness of SDHD compared with existing typical hallucination detection methods?
RQ2: What is the impact of the different modules of SDHD?
RQ3: What is the impact of the test cases expansion strategies for SDHD?
RQ4: What is the improvement of static detection over AST analysis and AST+CFG analysis?
RQ5: What is the effectiveness of SDHD in detecting different types of hallucinations?

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

C. Variables

1) Independent variables – approaches being evaluated: To facilitate a comprehensive comparison, we selected three baseline techniques for RQ1, which are CodeHalu [17], Selfcheck [22] and SCA3 [23]. CodeHalu is a dynamic detection algorithm designed to identify and quantify code hallucinations in LLMs during code generation. SelfCheck is a self-consistency paradigm that detects hallucinations by measuring the semantic alignment between the generated text and the input context. Its pipeline first samples a set of candidate continuations and then inspects whether their internal semantics remain mutually coherent; the degree of self-agreement serves as a proxy for factual reliability. This introspective strategy is particularly powerful against factuality hallucinations, where the model fabricates information that is not grounded in the prompt. SAC3 goes one step further by explicitly modelling the semantic relationship between the response and the original context. A pretrained language model encodes both pieces of text into dense vectors; the cosine similarity of these representations acts as a consistency score. Because SAC3 is sensitive to deviations in meaning rather than surface form, it excels at spotting faithfulness hallucinations, cases where the answer drifts from the user, intended message even though it may still appear fluent.

All baseline methods strictly follow the original paper settings: CodeHalu uses the default test cases; Selfcheck and SCA3 use GPT-3.5. Hyperparameters are not tuned to ensure a fair comparison. The software includes the Pycharm platform and OpenAI's GPT-3.5 API.

In the context of RQ2, We conducted experiments using both the static and dynamic detection methods individually and also evaluated their combined effects.

For RQ3, we investigated whether the test cases generated through the TDD approach impact the effectiveness of hallucination detection.

For RQ4, we conducted a comparative analysis of SSA against the use of AST alone and AST+CFG for hallucination detection.

For RQ5, we conducted a comparative analysis of SDHD on different types of hallucinations.

2) Dependent variables – evaluation metrics: For the evaluation of RQ1 through RQ5, we selected three key indicators: precision (P), recall (R), and the F1-score (F1). To comprehensively analyze the effectiveness of SDHD, we selected GPT-3.5 [24]. The experimental evaluation was carried out using four NVIDIA A6000 GPUs. Each model shared similar parameter settings: a temperature of 0.7 for balanced creativity and coherence, a maximum length of 1024 tokens for computational efficiency, and a top-p parameter of 1 for comprehensive token consideration. These settings ensured diverse, contextually relevant responses and effective hallucination detection without excessive complexity.

V. Results and Analysis

A. Answer to RQ1

The experimental results, presented in Table III, highlight the superior performance of SDHD across three datasets, where it achieved an F1-score exceeding 0.77 and demonstrated the best detection capabilities. In comparison to CodeHalu, SDHD shows significant improvements across all metrics—Precision, Recall, and F1-score—and across all datasets (MBPP, CHE, and HC). Notably, SDHD achieves an F1-score range of 0.776-0.778, while CodeHalu's F1-score only ranges from 0.482 to 0.496. SDHD's ability to identify hallucination issues extends to syntax, semantics, and logic. Moreover, by integrating SSA and expanding test cases, SDHD can detect hallucinations on each branch of the output, providing specific line numbers and pinpointing the exact location of hallucinatory information, thereby offering a more precise and targeted detection approach.

TABLE III
Hallucination Detection Result

Model	Metric	MBPP	CHE	HC
CodeHalu	P	0.445	0.457	0.452
	R	0.557	0.530	0.514
	F1	0.496	0.492	0.482
Selfcheck	P	0.160	0.122	0.140
	R	0.270	0.223	0.250
	F1	0.201	0.157	0.176
SCA3	P	0.278	0.240	0.300
	R	0.389	0.360	0.333
	F1	0.324	0.288	0.316
SDHD	P	**0.775**	**0.770**	**0.768**
	R	**0.778**	**0.780**	**0.790**
	F1	**0.776**	**0.773**	**0.778**

Furthermore, when compared to Selfcheck and SCA3, methods originating from the natural language domain, SDHD's F1-score (0.776-0.778) markedly surpasses those of Selfcheck (0.157-0.201) and SCA3 (0.288-0.324). This significant difference in performance across all datasets underscores SDHD's superior effectiveness in detecting hallucinations. While Selfcheck and SCA3 rely on consistency for judgment, the diverse nature of code generated by LLMs limits the effectiveness of these approaches in detecting code hallucinations.

The Selfcheck and SCA3 approaches rely on consistency to make judgments, but the diversity of code generated by LLMs renders the detection effectiveness of these two approaches inadequate for the detection of code hallucinations. In cases of detection, they can only identify code with hallucinatory elements, but they do not provide detailed information about the specific hallucinations. Our proposed detection framework integrates static and dynamic dual-route detection to identify eight types of syntactic, semantic, and logical hallucinations and exhibits good detection results across all three datasets.

In conclusion, SDHD demonstrates significant superiority over existing methods in detecting code generation hallucinations. It not only identifies syntactic and semantic errors but also leverages the integration of static and dynamic analysis to detect a broader spectrum of hallucination types. SDHD's test case expansion strategy and SSA form analysis further enhance its detection capabilities, enabling it to pinpoint

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
A COMPARISON BETWEEN STATIC AND DYNAMIC METHODS

Model	Metric	MBPP	CHE	HC
Static-only	P	0.473	0.467	0.500
	R	0.487	0.453	0.444
	F1	0.480	0.465	0.471
Dynamic-only	P	0.750	0.720	0.731
	R	0.700	0.700	0.739
	F1	0.724	0.693	0.735
SDHD	P	**0.775**	**0.770**	**0.768**
	R	**0.778**	**0.780**	**0.790**
	F1	**0.776**	**0.773**	**0.778**

hallucinations on each branch of the output and provide precise location information.

B. Answer to RQ2

The results are presented in Table IV. To underscore the necessity of our integrated static and dynamic detection approach, we conducted separate experiments with each module, aiming to observe their individual impact on detection outcomes. Our findings revealed that the performance of each module in isolation was inferior to their combined efficacy. This discrepancy arises because static detection, while particularly adept at identifying semantic and syntactic hallucinations, falls short in addressing certain logical hallucinations. In contrast, dynamic detection shines in uncovering logical hallucinations LDH and LFH, thanks to its reliance on high-coverage test cases. Consequently, static and dynamic detection methods complement each other to a significant extent, synergistically enhancing their overall detection capabilities.

Static analysis, while robust in identifying syntactic and semantic issues, lacks the capability to detect logical hallucinations that only manifest during code execution. Its reliance on code structure and syntax makes it less effective in scenarios where hallucinations are context-dependent or involve complex logic that isn't apparent statically.

Dynamic analysis is constrained by the scope and quality of test cases. If the test cases do not cover all possible execution paths, some hallucinations may remain undetected. In addition, dynamic execution can be resource-intensive and time-consuming, particularly for large codebases. This limitation can hinder its effectiveness in real-world applications where efficiency is crucial. Dynamic detection is primarily employed to identify logical hallucinations LDH and LFH. However, since DCH, SAH, IH, ESH, PCH, and CBH can also contribute to the occurrence of logical hallucinations, the effectiveness of dynamic detection tends to surpass that of static methods.

The SDHD approach leverages the complementary strengths of both static and dynamic analysis. The integration of both methods allows SDHD to address a broader spectrum of hallucination types. This comprehensive coverage results in higher precision (P), recall (R), and F1-score across all datasets (MBPP, CHE, HC), as evidenced by the table. The synergistic effect of combining static and dynamic detection methods leads to a significant enhancement in overall detection capabilities.

Static and dynamic analysis are complementary approaches to hallucination detection, each with its strengths and weaknesses. The choice of method should be based on the specific context and requirements of the project. In the SDHD framework, the combination of static and dynamic analysis significantly enhances the detection of code hallucinations, demonstrating the value of using both methods in tandem.

We also evaluated the computational cost of both static and dynamic analyses in the SDHD framework. For the static SSA analysis, MBPP, with an average of only 24 tokens per sample, is the fastest dataset to process, requiring approximately 3–10 ms per record. HalluCode contains moderately longer snippets and therefore takes around 10–25 ms per sample. In contrast, CodeHaluEval includes substantially longer and more complex programs, leading to a significantly higher analysis cost of approximately 40–120 ms per sample.

The dynamic analysis in SDHD primarily consists of test case generation and test execution. For test case generation, we measured the time required to produce one set of test cases for each program under various deployment configurations of LLMs. Because the generation step entails a full forward pass with structured reasoning, the latency varies considerably depending on hardware capabilities and whether the model is served locally or accessed via remote APIs. Our empirical results indicate that provider-hosted model APIs (e.g., OpenAI, DeepSeek, etc.) offer the lowest latency, allowing each test case to be processed in under 3 seconds due to optimized inference pipelines and high-performance accelerator infrastructure.

For test execution, the overhead is minimal. In a standard CPU environment, running a single program takes approximately 1–5 ms for MBPP, 3–8 ms for HalluCode, and 6–30 ms for CodeHaluEval, depending on the size and structural complexity of the code.

In conclusion, the synergistic collaboration between SDHD's static and dynamic detection modules surpasses the effectiveness of each module used independently. Static analysis excels at identifying syntactic and semantic errors, while dynamic analysis excels at detecting logical errors. This combination allows for a more comprehensive identification of hallucination types in code, thereby improving the accuracy and efficiency of detection.

C. Answer to RQ3

The results of this experiment, as shown in Table V, indicate that the expansion of test cases can further enhance the detection results. Given that code generation hallucination detection is driven solely by user requirements, the TDD approach is more aligned with the scenario of test case expansion. Compared to the original test cases provided by the experimental dataset and the minimal test set that satisfies the TDD method, we have discovered that appropriately increasing the number of test cases can detect more hallucinations.

A larger number of test cases enhances the coverage of different code paths and scenarios, thereby increasing the likelihood

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

TABLE V
AN ABLATION STUDY ABOUT THE NUMBER OF TEST CASES

Model	Metric	MBPP	CHE	HC
N <3	P	0.443	0.452	0.490
	R	0.542	0.514	0.479
	F1	0.487	0.485	0.482
$3 \leq N < 10$	P	0.466	0.491	0.521
	R	0.496	0.494	0.523
	F1	0.511	0.492	0.522
$10 \leq N \leq 15$	P	**0.748**	**0.725**	**0.733**
	R	**0.702**	**0.706**	**0.727**
	F1	**0.724**	**0.715**	**0.730**

TABLE VI
COMPARATIVE EXPERIMENTAL RESULTS AMONG AST, AST+CFG AND STATIC DETECTION

Model	Metric	MBPP	CHE	HC
AST	P	0.290	0.270	0.320
	R	0.320	0.340	0.330
	F1	0.304	0.303	0.325
AST+CFG	P	0.314	0.306	0.334
	R	0.309	0.363	0.353
	F1	0.311	0.332	0.343
SSA	P	**0.473**	**0.467**	**0.500**
	R	**0.487**	**0.453**	**0.444**
	F1	**0.480**	**0.465**	**0.471**

Note: The AST representative only converts the code into AST tree form for hallucination detection.

of identifying subtle hallucinations that might be missed with a smaller set of tests. More test cases introduce a broader range of user requirements and edge cases, which helps in uncovering hallucinations that may only manifest under specific conditions. With an expanded test set, the validation process becomes more robust, as it can better simulate real-world usage and uncover potential issues that are not apparent with minimal testing. Additionally, the time and resources expended on these newly added test cases differ only by seconds compared to the minimal test set required for the TDD method.

However, adding more test cases results in diminishing returns. The marginal improvement in detection metrics may not justify the additional time and resources required for creating and executing these tests. An excessively large number of test cases can introduce unnecessary complexity, making it harder to manage and maintain the test suite. This can lead to inefficiencies and potential errors in the testing process. More test cases require more computational resources and time for execution. This can be particularly problematic in resource-constrained environments, where the cost of running extensive tests may outweigh the benefits.

In conclusion, expanding test cases through the TDD method effectively enhances the detection of hallucinations in code generation. More test cases provide broader coverage of code paths and scenarios, increasing the likelihood of identifying subtle hallucinations that might be missed with a smaller test set.

D. Answer to RQ4

We used only the AST structure for detection. The results of this experiment are shown in Table VI, where the comparison of the results is quite apparent. During the experiment, we found that two types of hallucinations cannot be detected by the AST structure: Physical Constraint Hallucinations (PCH) and Computational Boundary Hallucinations (CBH), which belong to resource-based hallucinations and must be combined with data flow and control flow analysis during dynamic generation for detection.

In addition, we can observe that our detection framework generally outperforms the baseline model. The reason for this superiority lies in the fact that our static detection method is a combination of control flow and data flow dual-mode analysis, enabling it to detect both semantic and syntactic errors.

In conclusion, SSA form is highly effective in detecting hallucinations in code generation. It enables tracking of data flow, identifying variables with unexplained assignments, and detecting anomalies in the program. SSA analysis effectively identifies syntactic and semantic errors, as well as other types of hallucinations, such as structural access errors and naming errors.

E. Answer to RQ5

As demonstrated in Table VII, SDHD demonstrates a clear performance superiority in hallucination detection. The framework achieves superior efficacy on DCH, ESH, and LDH by leveraging the structural regularity inherent in partially correct code, generating localized errors and execution semantic deviations that produce detectable density homogeneity signatures. These types achieve F1-scores of over 0.800 across all datasets. By contrast, SAH, IH, PCH, CBH, and particularly LFH exhibit markedly inferior detection rates, with LFH consistently scoring below 0.410, as complete logical failures produce code devoid of meaningful structural regularities that fall outside the framework's detection scope.

This performance paradox reveals that while SDHD is intrinsically suited for identifying deviations within coherent solutions and inadequate for total breakdowns, its overall detection capability is superior to the state-of-the-art techniques. The SDHD's ability to capture structural anomalies in partially correct code provides significant value over existing methods, establishing its practicality while simultaneously underscoring the critical necessity for complementary detection methodologies to achieve comprehensive coverage across the full spectrum of code generation failures.

In conclusion, SDHD identifies structural regularity patterns in partially correct code to effectively detect DCH, ESH, and LDH hallucinations through semantic density analysis, while struggling with complete logical failures such as LFH that lack discernible patterns.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
EXPERIMENTAL RESULTS ON DIFFERENT TYPES OF HALLUCINATIONS

Dataset	Metric	MH			NH	RH		LH	
		DCH	SAH	IH	ESH	PCH	CBH	LDH	LFH
MBPP	P	0.744	0.467	0.403	0.793	0.511	0.440	**0.815**	0.363
	R	**0.860**	0.491	0.428	0.751	0.753	0.403	0.826	0.369
	F1	0.798	0.479	0.416	0.771	0.608	0.421	**0.820**	0.366
CHE	P	0.732	0.481	0.440	0.800	0.501	0.451	**0.801**	0.410
	R	**0.844**	0.455	0.460	0.691	0.740	0.433	0.833	0.409
	F1	0.781	0.467	0.450	0.743	0.597	0.442	**0.817**	0.409
HC	P	0.736	0.438	0.411	0.780	0.514	0.439	**0.806**	0.397
	R	0.722	0.466	0.405	0.693	0.503	0.404	**0.811**	0.361
	F1	0.729	0.450	0.406	0.734	0.508	0.421	**0.809**	0.379

F. Threats to Validity

Our study faces two primary threats to validity. Firstly, there is an internal threat related to our implementation. We reproduced CodeHalu, Selfcheck, and SCA3, following the procedures outlined in the original literature. Despite our efforts, some discrepancies in results may still arise. To mitigate this, we have rigorously verified the implementations of these methods through multiple rounds of checks by different individuals, ensuring their correctness and minimizing any discrepancies. Secondly, a threat to conclusion validity lies in the limited scope of our study. We have only applied the SDHD technique to the detection of hallucinations in Python code. To strengthen the generalizability of our findings, it is crucial for future work to extend the application of this method to other programming languages where hallucinations may occur.

VI. DISCUSSION

A. Static vs. Dynamic Hallucination Detection

It is critical to conduct a comparative analysis against the existing hallucination detection methods, both static and dynamic, to justify the superiority of our SDHD approach. Intuitively, SDHD goes beyond a simple head-to-head matchup and instead deconstructs the comparison into three distinct layers: comparison with existing state-of-the-art approaches, comparison of the fundamental paradigms (static vs. dynamic), and comparison of the underlying static analysis techniques.

Firstly, as detailed in our response to RQ1 and presented in Table III, we benchmarked SDHD against the most relevant existing methods, including the dynamic detection algorithm CodeHalu and methods from the natural language processing domain like SelfCheck and SCA3. The results are unequivocal: SDHD achieved an F1-score ranging from 0.776 to 0.778 across three diverse datasets, substantially outperforming all baselines. Notably, its performance was higher than that of CodeHalu. This provides definitive evidence that our hybrid approach is significantly more effective than a purely dynamic method or generic NLP-based consistency checks.

Secondly, to address the core static vs. dynamic debate, our analysis for RQ2 compared the performance of SDHD's individual modules against their integrated form. The results clearly demonstrate that while dynamic-only detection outperforms static-only detection, the synergistic combination in SDHD yields the highest overall performance. This comparison definitively shows that static and dynamic methods are not mutually exclusive but are, in fact, complementary. The hybrid framework takes full advantage of both static analysis for syntactic/semantic errors and dynamic analysis for logical flaws, consequently achieving a comprehensive detection capability that neither paradigm can match alone.

Finally, to validate our specific technical choices, we conducted a granular comparison for RQ4 between our SSA-based static detection and simpler static analysis methods like AST and AST+CFG. The results show that our SSA-based approach is significantly more effective, proving that the quality of static analysis is as important as its application. This comparison provides a definitive justification for our design, moving beyond the general "static analysis is useful" argument to demonstrate that our specific implementation is superior.

In conclusion, through these systematic and rigorous comparisons, this study provides a definitive and multi-faceted evaluation of SDHD against prior methods. We have not only shown that SDHD outperforms existing techniques but have also systematically elaborated the reasons behind the outperformance, by validating the hybrid paradigm and the specific SSA-based implementation.

B. Code Hallucination Types

Code hallucinations, which are defects in code generated by LLMs that appear plausible but are actually flawed, can manifest in various forms, each with distinct characteristics and detection challenges. These hallucinations can be broadly categorized into several common types. Syntactic hallucinations involve structurally correct code that contains syntax errors, like missing semicolons or mismatched parentheses, which are typically easy to find with static analysis tools. Semantic hallucinations are more subtle, involving flawed code logic that does not align with the intended functionality, such as incorrect variable assignments or misuse of control flow, and their detection often requires a combination of static and dynamic techniques for deep analysis. Logical hallucinations refer to code that seems to run correctly but produces unexpected results due to flawed

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

reasoning or assumptions, making their identification extremely challenging and often requiring extensive testing and domain expertise. Additionally, there are resource hallucinations involving improper resource management, contextual hallucinations where the broader environment is ignored, and domain-specific hallucinations relevant to a particular application area.

To provide a more systematic and granular classification system, the SDHD framework categorizes code hallucinations into eight specific types based on their nature and origin. These eight types include: Mapping Hallucination (MH), which concerns the incorrect mapping of data types, values, or structures; Naming Hallucination (NH), related to errors in variable, attribute, or module names; Resource Hallucination (RH), referring to an inadequate perception of resource consumption and control flow; and Logical Hallucination (LH), which manifests as differences between expected and actual results or low-density output semantics. Furthermore, the framework includes Data Compliance Hallucination (DCH), a failure to handle data type or format requirements; Structure Access Hallucination (SAH), errors in accessing complex data structures; Identity Hallucination (IH), a failure to correctly identify or reference variables, functions, or modules; and External Source Hallucination (ESH), errors in referencing external libraries or modules.

The eight types proposed by the SDHD framework are more specific and granular than the general categories, with their core advantage being a focus on the root causes of code hallucinations. For example, the "Mapping Hallucination (MH)" in the framework actually encompasses both "Data Compliance Hallucination" and "Structure Access Hallucination" from the general classification, as it uniformly addresses errors that occur when data is transformed between different forms or structures. Similarly, "Naming Hallucination (NH)" covers both "Identity Hallucination" and "External Source Hallucination," as they are fundamentally errors related to references and identifiers. This finer distinction enables developers to design more targeted detection tools and mitigation strategies, thereby locating and fixing specific types of code defects with greater precision.

It is important to note that these categories of code hallucinations are not always mutually exclusive; in practice, a single error can span multiple categories. This overlap adds to the complexity of the problem. For instance, a logical hallucination might stem from a deeper semantic error, where flawed logical reasoning is caused by a misunderstanding of the code's intended functionality. Likewise, a resource hallucination, such as a memory leak, could ultimately lead to a program logic error or system crash. Therefore, when analyzing and fixing code, a holistic perspective is needed to understand the potential connections and causal relationships between different types of hallucinations, rather than viewing each problem in isolation.

Ultimately, understanding the different types of code hallucinations and their interconnections is essential for developing effective detection and mitigation strategies. The severity of code hallucinations varies widely, with some causing only minor inconveniences while others lead to catastrophic failures in critical systems. Consequently, detecting these hallucinations, especially the elusive logical ones, remains a significant challenge. Combining various techniques, including both static and dynamic analysis, is key to achieving comprehensive detection. The SDHD framework and its hybrid analysis approach represent a significant step forward in tackling this challenge and improving the reliability of code generated by LLMs.

C. Hallucination Detection in Large-Scale Code Generation

We conducted experiments on three project-level case studies: an E-Commerce System (ECS)[2], a Campus Security Management System (CSMS)[3], and a Stock Trading System (STS)[4]. Our results reveal a fundamental scalability challenge: code generation is often structurally incomplete at the project level, which significantly impedes SDHD's ability to detect hallucinations. Specifically, when generated classes lack complete inheritance hierarchies, method implementations, or inter-component dependencies, SDHD's static analysis module cannot construct complete control-flow and data-flow graphs, while its dynamic testing module fails to generate executable test harnesses for isolated components. This incompleteness creates detection shadows where hallucinated APIs embedded in unfinished code paths evade automated scrutiny, representing a critical limitation that function-level evaluation frameworks do not capture.

Through systematic manual code review of these generated systems, we identified pervasive API hallucinations [25] that automated tools alone cannot reliably detect in large-scale contexts. Across the three projects, numerous instances emerged where LLMs fabricated method signatures, incorrectly overloaded existing APIs, or invoked non-existent methods from third-party libraries. These hallucinations frequently reside in exception-handling branches or configuration modules that bypass standard test coverage, requiring human expertise to identify semantic inconsistencies that evade both static syntax checks and dynamic execution metrics.

These empirical findings position API hallucination as a paramount problem in large-scale code generation and a critical direction for future research. Our project-level evidence revealed that hallucination patterns become more sophisticated and context-dependent as codebases scale, necessitating enhanced analysis that considers architectural coherence and cross-component dependencies. Future work is needed to develop augmented detection frameworks that combine fine-grained hybrid analysis with project-level dependency graphs and repository-wide API usage patterns, potentially integrating retrieval-augmented generation techniques that prioritize external API documentation over parametric knowledge when conflicts arise, as suggested by the knowledge conflict mitigation strategies identified in prior research.

D. Hallucinations and Context

Insufficient context is the primary cause of hallucinations, a problem that manifests in our work as structural incompleteness and pervasive API errors. When an LLM lacks a comprehensive view of a project's architecture and dependencies, it tends

[2]https://github.com/macrozheng/mall
[3]https://github.com/vinaysingh5898/Online-Campus-Security-Management
[4]https://github.com/Krushna153/Stock-Trading-Software

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

to fabricate code structures, creating the detection shadows our analysis revealed. To proactively address this fundamental issue, Retrieval-Augmented Generation (RAG) emerges as a powerful preventative strategy [26], [27]. By dynamically retrieving relevant project-level context, such as up-to-date API documentation and concrete usage examples, RAG provides the LLM with the necessary grounding to generate structurally complete and semantically correct code, thereby mitigating hallucinations at their source.

However, even with rich context, hallucinations persist due to three root causes: (1) Context defects: retrieved context may be outdated, ambiguous, or incomplete, causing LLMs to fabricate solutions; (2) Knowledge conflicts: parametric knowledge in LLMs can contradict retrieved context, leading to semantic errors; (3) Logical complexity: certain hallucinations (e.g., LFH) stem from flawed reasoning rather than context gaps. As shown in Table VII, SDHD achieves F1 < 0.41 for LFH, proving that context alone cannot resolve fundamental logical faults.

In addition, prevention is not infallible. Conflicts between retrieved context and the LLM's parametric knowledge, or fault in the retrieval process itself, can still lead to errors. Therefore, robust detection mechanisms like our proposed SDHD remain indispensable as a critical quality assurance approach. Consequently, the most effective way lies in a synergistic approach. Future work should focus on developing integrated frameworks where RAG supplies rich context to prevent hallucinations, and advanced detectors like SDHD identify any residual errors that slip through. This creates a comprehensive pipeline for reliable large-scale code generation, combining proactive prevention with reactive detection to ensure the highest possible quality and correctness.

VII. Related Work

A. Code Generation

Code generation is a process of automatically generating source code based on functional requirements (e.g., natural language descriptions) [28], [29], [30], [31], [32] or pseudocode algorithms [33], [34] or responses from programming tools [35]. The current mainstream code generation task is focused on method-level generation, both function-level and competitively turned-level. In recent years, large pre-trained language models such as AlphaCode [36] and GPT3 [37] series, have demonstrated impressive capabilities in code generation and competitive programming. Other open source code generation models include GPT-Neo [38], GPT-J [39], CodeParrot [40], CodeGen [41], StarCoder [42], and DeepSeek [43]. The AlphaCode system [36] demonstrated the competitiveness of AI in programming competitions on the Codeforces platform by showing a level of competition with the top 54.3% after generating and filtering a large number of different programs to solve problems. Based on the aspect of executing test cases, Liu [44] proposes EvalPlus, a code synthesis evaluation framework that rigorously evaluates the functional correctness of code generated by LLMs through a large number of newly generated test cases. More so, a self-collaborative code generation framework [45] using a large language model (e.g., ChatGPT) achieves a performance improvement on multiple code generation benchmarks.

B. Hallucination Detection in Natural Language Processing

Correct GQA (Generate-QA) aims at generating answers that are strictly based on source text or valid external knowledge, and there has been more research focused on this [46], [47], [48]. The more faithful the generated answer is, the less illusory content it contains. Semantic drift (a phenomenon in which the meaning of a word or phrase changes over time. The semantic meaning of a word may expand, contract, or change completely in different contexts or over time), factual correctness, and other terms can also reflect the level of hallucination. The evidence answer generator (REAG) [49] adds an extraction task to obtain evidence for answers and generates high-confidence answers. Read-before-Generate [47] combines answer generation with machine reading to integrate fine-grained, salient information relevant to the answer. The benchmark proposed by Lin [50] measures the truthfulness of answers generated by language models from different domains. LLM-Augmenter [51] improves LLMs with external knowledge and automatic feedback using a PnP module, which does not require any training and is plug-and-play. Based on user queries, the framework first retrieves evidence from external knowledge and performs reasoning to form an evidence chain. The D&Q framework [52] guides models that utilize external knowledge while restricting inference to reliable information, thus mitigating the risk of hallucinations. The framework involves a supervised fine-tuning phase without tool calls, and in the prediction phase, the model uses external tools to query a reliable Q&A base, allowing for backtracking and initiating new searches when needed. Other researchers, who argue that one can rely on the self-consistency of the model to detect hallucinations [22], [23], have argued that if the LLM understands given concepts, the sampled responses are likely to be similar and contain consistent facts. However, for hallucinatory facts, randomly sampled responses may diverge and contradict each other. However, for hallucinatory facts, randomly sampled responses may diverge and contradict each other.

C. Code Generation Hallucination Detection and Mitigation

LLMs have revolutionized the field of code generation, enabling the automatic creation of code snippets based on natural language descriptions. However, the rise of LLMs has also exposed a significant risk: code generation hallucinations. This phenomenon refers to the generation of code snippets that seem plausible but deviate from user intent, context, or established facts, potentially leading to incorrect functionality, security vulnerabilities, and other issues.

Several studies have proposed taxonomies to categorize different types of code generation hallucinations. For example, Liu et al. [16] identified five main categories: intent conflicting, context inconsistency, context repetition, dead code, and knowledge conflicting. Zhang et al. [18] further refined these

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

categories into three broader categories: task requirement conflicts, factual knowledge conflicts, and project context conflicts.

Researcher has explored various techniques to detect code generation hallucinations. Tian et al. [17] introduces the concept of code hallucinations and proposes a classification method based on execution verification. It categorizes hallucinations into four main types: mapping, naming, resource, and logic hallucinations, with each further divided into subcategories. The study also introduces CodeHalu, a dynamic detection algorithm designed to identify and quantify code hallucinations, along with the CodeHaluEval benchmark, which includes a diverse set of tasks and samples to evaluate hallucinations in LLMs. However, current methods still struggle with detecting subtle or hidden types of hallucinations.

To address the risk of code generation hallucinations, researchers have proposed several mitigation strategies [26], [27], [53], [54], [55]. These include RAG [56], which provides LLMs with relevant code snippets from a retrieval library to enhance their understanding of the task and context; input query refinement [57], which refines user input to better align with the LLM's capabilities; and multi-agent systems [58], which involve iterative processes of clarifying requirements, generating code, testing, and refining to mitigate hallucinations. These studies emphasize the importance of reducing hallucinations, which is the basis of our work.

VIII. Conclusion

SDHD, a hybrid framework combining static and dynamic analysis, effectively addresses the growing concern of code hallucinations in LLM-generated code. By leveraging SSA forms for static analysis and a novel test case generation technique for dynamic testing, SDHD significantly improves detection accuracy and coverage compared to existing methods. This comprehensive approach allows for the identification of a broader range of hallucinations, including both syntactic and semantic errors, leading to more reliable and trustworthy code generation.

SDHD's strength lies in its ability to combine the benefits of both static and dynamic analysis. While static analysis is effective in identifying semantic and syntactic issues, dynamic testing uncovers logical hallucinations that may only manifest during code execution. This complementary approach results in a synergistic effect, enhancing overall detection capabilities and achieving superior performance on various datasets.

In future work, we will address the primary root cause of hallucinations, namely the insufficient context, through two complementary activities: prevention and detection. On one hand, to proactively prevent hallucinations, we will integrate Retrieval-Augmented Generation (RAG) to provide richer, project-specific context, thereby reducing hallucinations at the source. On the other hand, to strengthen our detection capabilities, we will systematically port the SDHD hybrid core (SSA-based static scanner plus differential dynamic executor) from Python to mainstream languages including Java and C++, ensuring that the iterative "generate-test-refine" loop, which continuously strengthens test suites and suppresses subtle hallucinations, remains intact while broadening the applicability and practical impact of our approach across diverse software ecosystems.

Acknowledgment

We would like to express our sincere gratitude to Dr. Fang Liu from Beihang University for generously providing the related datasets.

References

[1] E. Kamalloo, S. Upadhyay, and J. Lin, "Towards robust QA evaluation via open LLMs," in *Proc. 47th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2024, pp. 2811–2816.

[2] J. Yang et al., "Harnessing the power of LLMs in practice: A survey on ChatGPT and beyond," *ACM Trans. Knowl. Discovery Data*, vol. 18, no. 6, pp. 1–32, 2024.

[3] B. Steenhoek, H. Gao, and W. Le, "Dataflow analysis-inspired deep learning for efficient vulnerability detection," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, Lisbon, Portugal. New York, NY, USA: ACM, 2024, pp. 16:1–16:13, doi: 10.1145/3597503.3623345.

[4] C. Georgescu, M. Olsthoorn, P. Derakhshanfar, M. Akhin, and A. Panichella, "Evolutionary generative fuzzing for differential testing of the Kotlin compiler," in *Proc. Companion 32nd ACM Int. Conf. Found. Softw. Eng. (FSE)*, Porto De Galinhas, Brazil, M. d'Amorim, Ed. New York, NY, USA: ACM, 2024, pp. 197–207, doi: 10.1145/3663529.3663864.

[5] V. Adlakha, P. BehnamGhader, X. H. Lu, N. Meade, and S. Reddy, "Evaluating correctness and faithfulness of instruction-following models for question answering," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 681–699, May 2024.

[6] S. Biderman et al., "Pythia: A suite for analyzing large language models across training and scaling," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 2397–2430.

[7] J. Chen, L. Zhenhao, H. Xing, and X. Xin, "NLPerturbator: Studying the robustness of code LLMs to natural language variations," *ACM Trans. Softw. Eng. Methodol.*, Jul. 2024, doi: 10.1145/3745764

[8] Z. Zhao, Y. Cao, S. Gong, and I. Patras, "Enhancing zero-shot facial expression recognition by LLM knowledge transfer," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vision (WACV)*, Piscataway, NJ, USA: IEEE Press, 2025, pp. 815–824.

[9] M. J. Mirza et al., "Meta-prompting for automating zero-shot visual recognition with LLMs," in *Proc. Eur. Conf. Comput. Vision,* New York, NY, USA: Springer-Verlag, 2024, pp. 370–387.

[10] Q. Zhao, Y. Dai, H. Li, W. Hu, F. Zhang, and J. Liu, "LTGC: Long-tail recognition via leveraging LLMs-driven generated content," in *Proc. IEEE/CVF Conf. Comput. Vision Pattern Recognit.*, 2024, pp. 19510–19520.

[11] X. Du et al., "ClassEval: A manually-crafted benchmark for evaluating llms on class-level code generation," 2023, *arXiv:2308.01861*.

[12] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 36, pp. 21558–21572, Art. no. 943.

[13] Y. Dong, X. Jiang, Z. Jin, and G. Li, "Self-collaboration code generation via ChatGPT," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 7, pp. 1–38, 2024.

[14] J. Chen, Z. Pan, X. Hu, Z. Li, G. Li, and X. Xia, "Reasoning runtime behavior of a program with LLM: How far are we?" 2024, *arXiv:2403.16437*.

[15] J. Chen, Z. Li, Q. Mao, X. Hu, K. Liu, and X. Xia, "Understanding practitioners' expectations on clear code review comments," *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, pp. 1257–1279, 2025.

[16] F. Liu et al., "Exploring and evaluating hallucinations in LLM-powered code generation," 2024, *arXiv:2404.00971*.

[17] Y. Tian et al., "CodeHalu: Investigating code hallucinations in LLMs via execution-based verification," 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:269484644

[18] Z. Zhang, Y. Wang, C. Wang, J. Chen, and Z. Zheng, "LLM hallucinations in practical code generation: Phenomena, mechanism, and mitigation," 2024, *arXiv:2409.20550*.

[19] J. Austin et al., "Program synthesis with large language models," 2021, *arXiv:2108.07732*.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.

[20] E. Nijkamp et al., "CodeGen: An open large language model for code with multi-turn program synthesis," 2022, *arXiv:2203.13474*.
[21] H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi, "CodeRL: Mastering code generation through pretrained models and deep reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 21314–21328.
[22] M. Li, B. Peng, M. Galley, J. Gao, and Z. Zhang, "Self-checker: Plug-and-play modules for fact-checking with large language models," 2023, *arXiv:2305.14623*.
[23] P. Manakul, A. Liusie, and M. J. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," 2023, *arXiv:2303.08896*.
[24] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
[25] N. Jain, R. Kwiatkowski, B. Ray, M. K. Ramanathan, and V. Kumar, "On mitigating code LLM hallucinations with API documentation," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng., Softw. Eng. in Pract. (ICSE-SEIP)*, Piscataway, NJ, USA: IEEE Press, 2025, pp. 237–248.
[26] Y. Chen, M. Chen, C. Gao, Z. Jiang, Z. Li, and Y. Ma, "Towards mitigating API hallucination in code generated by LLMs with hierarchical dependency aware," in *Proc. 33rd ACM Int. Conf. Found. Softw. Eng.*, 2025, pp. 468–479.
[27] Z. Zhang et al., "LLM hallucinations in practical code generation: Phenomena, mechanism, and mitigation," *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, pp. 481–503, 2025.
[28] S. Iyer, I. Konstas, A. Cheung, and L. Zettlemoyer, "Mapping language to code in programmatic context," 2018. [Online]. Available: https://arxiv.org/abs/1808.09588
[29] P. Yin and G. Neubig, "TRANX: A transition-based neural abstract syntax parser for semantic parsing and code generation," 2018, *arXiv:1810.02720*.
[30] J. Li, G. Li, Y. Li, and Z. Jin, "Enabling programming thinking in large language models toward code generation," 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2305.06599
[31] J. Li et al., "CodeEditor: Learning to edit source code with pre-trained models," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 6, pp. 1–22, 2023.
[32] J. Li, Y. Li, G. Li, Z. Jin, Y. Hao, and X. Hu, "SKCODER: A sketch-based approach for automatic code generation," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 2124–2135.
[33] S. Kulal et al., "SPOC: Search-based pseudocode to code," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, vol. 32, pp. 11906–11917, Art. no. 1066.
[34] Y. Oda et al., "Learning to generate pseudo-code from source code using statistical machine translation," in *Proc. 30th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE Press, 2015, pp. 574–584.
[35] K. Zhang, H. Zhang, G. Li, J. Li, Z. Li, and Z. Jin, "ToolCoder: Teach code generation models to use API search tools," 2023, *arXiv:2305.04032*.
[36] Y. Li et al., "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
[37] T. B. Brown et al., "Language models are few-shot learners," 2020. [Online]. Available: https://arxiv.org/abs/2005.14165
[38] L. Gao et al., "The pile: An 800gb dataset of diverse text for language modeling," 2020. [Online]. Available: https://arxiv.org/abs/2101.00027
[39] B. Wang and A. Komatsuzaki, "GPT-J-6B: A 6 billion parameter autoregressive language model," 2021, *arXiv:2212.14815v3*.
[40] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empirical Methods Natural Lang. Process., Syst. Demonstrations*, 2020, pp. 38–45.
[41] E. Nijkamp et al., "A conversational paradigm for program synthesis," 2022, *arXiv:2203.13474*.
[42] R. Li et al., "StarCoder: May the source be with you!" 2023, *arXiv:2305.06161*.
[43] Q. Zhu et al., "DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence," 2024, *arXiv:2406.11931*.
[44] J. Liu, C. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:258437095
[45] Y. Dong, X. Jiang, Z. Jin, and G. Li, "Self-collaboration code generation via ChatGPT," *ACM Trans. Softw. Eng. Methodol.*, 2023. vol. 33, no. 7, p. 38, Sep. 2023, Art. no: 189. Available: https://api.semanticscholar.org/CorpusID:258179537
[46] R. Nakano et al., "WebGPT: Browser-assisted question-answering with human feedback," 2021, *arXiv:2112.09332*.
[47] D. Su et al., "Read before generate! Faithful long form question answering with machine reading," 2022. [Online]. Available: https://arxiv.org/abs/2203.00343
[48] D. Su et al., "Context generation improves open domain question answering," 2023. [Online]. Available: https://arxiv.org/abs/2210.06349
[49] C. Li, B. Bi, M. Yan, W. Wang, and S. Huang, "Addressing semantic drift in generative question answering with auxiliary extraction," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics 11th Int. Joint Conf. Natural Lang. Process. (Volume 2: Short Papers)*, 2021, pp. 942–947.
[50] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring how models mimic human falsehoods," 2021, *arXiv:2109.07958*.
[51] B. Peng et al., "Check your facts and try again: Improving large language models with external knowledge and automated feedback," 2023, *arXiv:2302.12813*.
[52] H. Cao, Z. An, J. Feng, K. Xu, L. Chen, and D. Zhao, "A step closer to comprehensive answers: Constrained multi-stage question decomposition with large language models," 2023, *arXiv:2311.07491*.
[53] V. Agarwal, Y. Pei, S. Alamir, and X. Liu, "CodeMirage: Hallucinations in code generated by large language models," 2024, *arXiv:2408.08333*.
[54] A. Eghbali and M. Pradel, "De-hallucinator: Mitigating LLM hallucinations in code generation tasks via iterative grounding," 2024, *arXiv:2401.01701*.
[55] J. Kim, H. Kim, K. Yeonju, and Y. M. Ro, "Code: Contrasting self-generated description to combat hallucination in large multi-modal models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 37, pp. 133571–133599.
[56] F. Zhang et al., "RepoCoder: Repository-level code completion through iterative retrieval and generation," 2023, *arXiv:2303.12570*.
[57] F. Mu et al., "ClarifyGPT: Empowering LLM-based code generation with intention clarification," 2023, *arXiv:2310.10996*.
[58] T. Guo et al., "Large language model based multi-agents: A survey of progress and challenges," 2024, *arXiv:2402.01680*.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on February 26,2026 at 06:36:38 UTC from IEEE Xplore. Restrictions apply.